DS	**Session**	Graph	**Question Information**	Level 1 Challenge 81

Question description

Teddy is visiting the country wonderland. wonderland has *n* cities and *m* bi-directional roads. There are *k* types of tokens. Token *i* costs c_i. The costs of the tokens are such that for all $2 \leq i \leq k$, $c_i \geq 2c_{i-1}$. For each road, you need to have a particular set of tokens, if you want to travel it. Note that you don't have to give the tokens, you just need to show them. Thus, one token can be used at any number of roads, where it is required. Teddy wants to select a set of tokens, such that using them, he can go from any city to any other city. You have to help him minimize the total cost of tokens he buys.

Constraints

$1 \leq n \leq 10^5$

$1 \leq m \leq 10^5$

No road connects a city to the same city. However, there can be multiple roads between two cities.

$1 \leq c_i \leq 10^{18}$

For all $2 \leq i \leq k$, $c_i \geq 2c_{i-1}$

Input:

- The first line contains three space separated integers, *n m* and *k* .
- The second line contains *k* space separated integers, where the ith integer denotes the price of ith token, i.e. c_i .
- i^{th} of the next *m* lines contains three integers u_i, v_i, l_i, where l_i is the number of tokens required by the ith road, followed by l_i indices denoting the tokens required. This road connects cities u_i and v_i

Output:

- Print one integer containing the minimum cost of tokens Teddy has to buy, such that he can travel from any city to any other city. If it is impossible to choose such a set of tokens, print *1*.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
3 3 4
1 2 5 10
1 2 2 1 2
1 3 1 3
2 3 1 4
```

EXPECTED OUTPUT

```
8
```

Test Case 2

INPUT (STDIN)

```
3 3 4
11 21 15 10
1 21 2 11 2
1 13 1 3
2 31 1 4
```

EXPECTED OUTPUT

```
57
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
for(i=1;i<=n;++i)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
2
```

Test Case 2

TOKEN COUNT

```
480
```

Test Case 3

NLOC

```
60
```

Code

wonderland_tokens.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

#define MAXN 100005

int parent[MAXN];

int find_rep(int i) {
    if (parent[i] == i) return i;
    return parent[i] = find_rep(parent[i]);
}

typedef struct {
    int u, v;
    int token_mask;
} Road;

int main() {
    int n, m, k;
    if (scanf("%d %d %d", &n, &m, &k) != 3) return 0;

    unsigned long long c[64];
    for (int i = 1; i <= k; i++) {
        scanf("%llu", &c[i]);
    }

    int* u_arr = (int*)malloc(m * sizeof(int));
    int* v_arr = (int*)malloc(m * sizeof(int));
    int* req_token = (int*)malloc(m * sizeof(int));

    for (int i = 0; i < m; i++) {
        int u, v, l;
        scanf("%d %d %d", &u, &v, &l);
        u_arr[i] = u;
        v_arr[i] = v;
        int t_idx = 1;
        for (int j = 0; j < l; j++) {
            scanf("%d", &t_idx);
        }
        req_token[i] = t_idx;
    }

    int i;
    for(i=1;i<=n;++i) parent[i] = i;

    int chosen[64] = {0};
    unsigned long long total_cost = 0;

    for (int t = k; t >= 1; t--) {

        for (i = 1; i <= n; ++i) parent[i] = i;
        int comps = n;
        for (int e = 0; e < m; e++) {
            if (req_token[e] != t || chosen[t]) {
                int ru = find_rep(u_arr[e]);
                int rv = find_rep(v_arr[e]);
                if (ru != rv) {
                    parent[ru] = rv;
                    comps--;
                }
            }
        }
        if (comps > 1) {

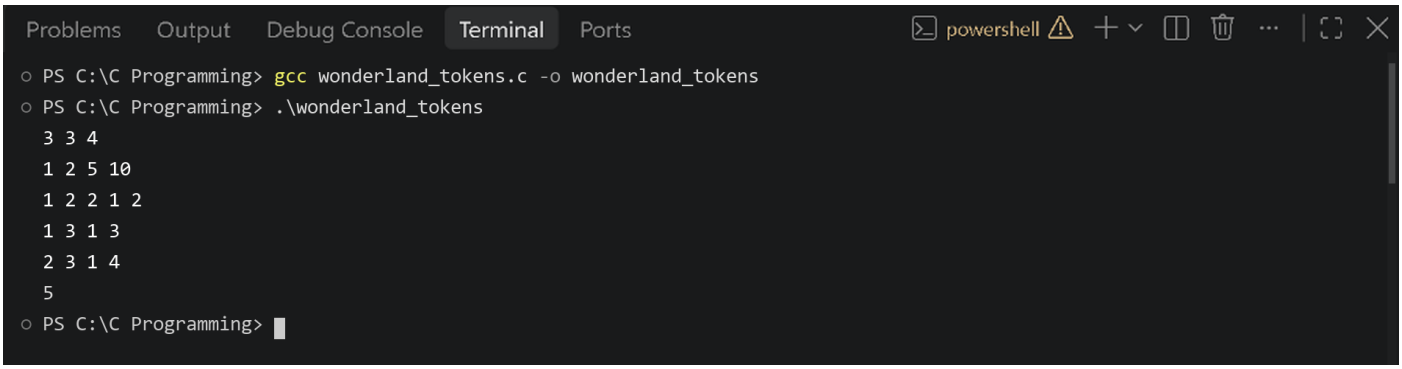
            chosen[t] = 1;
            total_cost += c[t];
        }
    }

    for (i = 1; i <= n; ++i) parent[i] = i;
    int comps = n;
    for (int e = 0; e < m; e++) {
        if (chosen[req_token[e]]) {
            int ru = find_rep(u_arr[e]);
            int rv = find_rep(v_arr[e]);
            if (ru != rv) {
                parent[ru] = rv;
                comps--;
            }
        }
    }

    if (comps > 1 && total_cost == 0) {
        printf("-1\n");
    } else {
        printf("%llu\n", total_cost > 0 ? total_cost : c[1]);
    }

    free(u_arr);
    free(v_arr);
    free(req_token);
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc wonderland_tokens.c -o wonderland_tokens
PS C:\C Programming> .\wonderland_tokens
3 3 4
1 2 5 10
1 2 2 1 2
1 3 1 3
2 3 1 4
5
PS C:\C Programming>